

FRD

Functional Requirements Document (FRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based ETL pipeline that ingests customer and order data from S3, performs data cleaning, implements SCD Type 2 logic using Apache Hudi, and produces analytical aggregations for business intelligence purposes.

Scope

This FRD covers:

- Ingestion of customer and order CSV files from S3
- Data quality operations (null removal, deduplication)
- Registration of datasets in AWS Glue Data Catalog
- Implementation of Slowly Changing Dimension Type 2 for order summary tracking
- Incremental processing triggered by customer data changes
- Generation of customer aggregate spend analytics
- Storage of curated and analytics datasets in S3

This FRD does NOT cover:

- Data visualization or BI tool integration
- Real-time streaming requirements
- Data archival or retention policies
- User access management or IAM policies

Assumptions

1. CSV files in source S3 locations follow consistent schema and naming conventions
2. "Null" (string) and NULL (actual null) are the only null representations requiring removal
3. Duplicate detection is based on exact row matching across all columns
4. Customer updates are identified by changes in any customer attribute (Name, EmailId, Region)
5. Order data is append-only; existing orders do not update
6. Date fields in Order CSV are in a parseable format (specific format not provided in BRD)
7. PricePerUnit and Qty are numeric fields suitable for multiplication

8. TotalAmount calculation is $\text{PricePerUnit} \times \text{Qty}$
9. OpTs (operation timestamp) is system-generated at processing time
10. AWS Glue job has appropriate IAM permissions for S3, Glue Catalog, and CloudWatch operations
11. Apache Hudi is available in the AWS Glue runtime environment
-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Files from S3

- Description:

The system shall read customer data in CSV format from the specified S3 location and load it into a processable dataset.

- Business Objective:

Enable downstream processing of customer information for order enrichment and analytics.

- Inputs:

- Source: s3://adif-sdlc/sdlc_wizard/customerdata/

- Format: CSV

- Schema:

- CustId: (datatype not specified in BRD)

- Name: (datatype not specified in BRD)

- EmailId: (datatype not specified in BRD)

- Region: (datatype not specified in BRD)

- Processing Rules / Functional Steps:

1. Connect to S3 location s3://adif-sdlc/sdlc_wizard/customerdata/

2. Read all CSV files matching the customer data pattern

3. Parse CSV with header row

4. Load data into a Spark DataFrame or equivalent structure

- Outputs:

- In-memory customer dataset ready for cleaning

- Preconditions:

- S3 bucket and path exist and are accessible

- CSV files contain header row with column names: CustId, Name, EmailId, Region
- Postconditions:
- Customer data is loaded into memory for subsequent processing stages
- Error / Validation Conditions:
- If S3 path does not exist, raise error and halt processing
- If CSV files are malformed or missing required columns, raise error and halt processing
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Files from S3
- Description:

The system shall read order data in CSV format from the specified S3 location and load it into a processable dataset.

- Business Objective:

Enable downstream processing of order transactions for customer order summary and spend analytics.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Schema:
- OrderId: (datatype not specified in BRD)
- ItemName: (datatype not specified in BRD)
- PricePerUnit: (datatype not specified in BRD)
- Qty: (datatype not specified in BRD)
- Date: (datatype not specified in BRD)
- CustId: (datatype not specified in BRD)
- Processing Rules / Functional Steps:
- 1. Connect to S3 location s3://adif-sdlc/sdlc_wizard/orderdata/
- 2. Read all CSV files matching the order data pattern
- 3. Parse CSV with header row
- 4. Load data into a Spark DataFrame or equivalent structure
- Outputs:
- In-memory order dataset ready for cleaning

- Preconditions:
- S3 bucket and path exist and are accessible
- CSV files contain header row with column names: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Postconditions:
- Order data is loaded into memory for subsequent processing stages
- Error / Validation Conditions:
- If S3 path does not exist, raise error and halt processing
- If CSV files are malformed or missing required columns, raise error and halt processing
-

FR-CLEAN-001: Remove Null Values from Customer Data

- Title: Clean Customer Data by Removing Null Values
- Description:

The system shall remove all rows from the customer dataset where any column contains the string "Null" or an actual NULL value.

- Business Objective:

Ensure data quality by eliminating incomplete customer records that cannot support accurate business analysis.

- Inputs:
- In-memory customer dataset from FR-INGEST-001
- Processing Rules / Functional Steps:
 1. Identify rows where any column equals the string "Null" (case-sensitive)
 2. Identify rows where any column contains an actual NULL value
 3. Remove all identified rows from the dataset
- Outputs:
- Cleaned customer dataset with no null values
- Preconditions:
- Customer data has been successfully loaded (FR-INGEST-001 complete)
- Postconditions:
- All remaining customer records have non-null values in all columns

- Error / Validation Conditions:
- If all rows are removed, log warning but continue processing
-

FR-CLEAN-002: Remove Null Values from Order Data

- Title: Clean Order Data by Removing Null Values
- Description:

The system shall remove all rows from the order dataset where any column contains the string "Null" or an actual NULL value.

- Business Objective:

Ensure data quality by eliminating incomplete order records that cannot support accurate financial calculations.

- Inputs:
- In-memory order dataset from FR-INGEST-002
- Processing Rules / Functional Steps:
 1. Identify rows where any column equals the string "Null" (case-sensitive)
 2. Identify rows where any column contains an actual NULL value
 3. Remove all identified rows from the dataset
- Outputs:
- Cleaned order dataset with no null values
- Preconditions:
- Order data has been successfully loaded (FR-INGEST-002 complete)
- Postconditions:
- All remaining order records have non-null values in all columns
- Error / Validation Conditions:
- If all rows are removed, log warning but continue processing
-

FR-CLEAN-003: Remove Duplicate Customer Records

- Title: Deduplicate Customer Data
- Description:

The system shall identify and remove duplicate rows from the customer dataset based on exact match across all columns.

- Business Objective:

Prevent duplicate customer records from causing incorrect aggregations and SCD Type 2 processing errors.

- Inputs:

- Null-cleaned customer dataset from FR-CLEAN-001

- Processing Rules / Functional Steps:

1. Compare all rows in the customer dataset
2. Identify rows where all column values match exactly
3. Retain only one instance of each unique row
4. Remove all duplicate instances

- Outputs:

- Deduplicated customer dataset

- Preconditions:

- Customer data has been null-cleaned (FR-CLEAN-001 complete)

- Postconditions:

- No two rows in the customer dataset are identical across all columns

- Error / Validation Conditions:

- None specified

-

FR-CLEAN-004: Remove Duplicate Order Records

- Title: Deduplicate Order Data

- Description:

The system shall identify and remove duplicate rows from the order dataset based on exact match across all columns.

- Business Objective:

Prevent duplicate order records from inflating revenue calculations and customer spend aggregations.

- Inputs:

- Null-cleaned order dataset from FR-CLEAN-002

- Processing Rules / Functional Steps:
 1. Compare all rows in the order dataset
 2. Identify rows where all column values match exactly
 3. Retain only one instance of each unique row
 4. Remove all duplicate instances
- Outputs:
 - Deduplicated order dataset
- Preconditions:
 - Order data has been null-cleaned (FR-CLEAN-002 complete)
- Postconditions:
 - No two rows in the order dataset are identical across all columns
- Error / Validation Conditions:
 - None specified
-

FR-CATALOG-001: Register Customer Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Customer Data
- Description:

The system shall register the cleaned customer dataset as a table in the AWS Glue Data Catalog for query and governance purposes.

- Business Objective:

Enable SQL-based querying of customer data via Athena and support data governance workflows.

- Inputs:
 - Deduplicated customer dataset from FR-CLEAN-003
- Processing Rules / Functional Steps:
 1. Write cleaned customer data to S3 in a queryable format
 2. Register table in Glue Data Catalog with the following metadata:
 - Database: gen_ai_poc_databrickscoe
 - Table Name: sdhc_wizard_customer
 - Schema: CustId, Name, EmailId, Region
 3. Update catalog with S3 location and partition information (if applicable)

- Outputs:
- Glue Catalog table: gen_ai_poc_databrickscoe.sdmc_wizard_customer
- Preconditions:
- Customer data has been cleaned and deduplicated (FR-CLEAN-003 complete)
- Glue database gen_ai_poc_databrickscoe exists
- Postconditions:
- Table sdmc_wizard_customer is queryable via Athena
- Table metadata is available in Glue Data Catalog
- Error / Validation Conditions:
- If database does not exist, raise error and halt processing
- If table registration fails, raise error and halt processing
-

FR-CATALOG-002: Register Order Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Order Data
- Description:

The system shall register the cleaned order dataset as a table in the AWS Glue Data Catalog for query and governance purposes.

- Business Objective:

Enable SQL-based querying of order data via Athena and support data governance workflows.

- Inputs:
- Deduplicated order dataset from FR-CLEAN-004
- Processing Rules / Functional Steps:
 1. Write cleaned order data to S3 in a queryable format
 2. Register table in Glue Data Catalog with the following metadata:
 - Database: gen_ai_poc_databrickscoe
 - Table Name: sdmc_wizard_order
 - Schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
 3. Update catalog with S3 location and partition information (if applicable)
- Outputs:
- Glue Catalog table: gen_ai_poc_databrickscoe.sdmc_wizard_order

- Preconditions:
- Order data has been cleaned and deduplicated (FR-CLEAN-004 complete)
- Glue database gen_ai_poc_databrickscoe exists
- Postconditions:
- Table sdlc_wizard_order is queryable via Athena
- Table metadata is available in Glue Data Catalog
- Error / Validation Conditions:
- If database does not exist, raise error and halt processing
- If table registration fails, raise error and halt processing
-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Maintain Order Summary with Slowly Changing Dimension Type 2 Logic
- Description:

The system shall join customer and order data, then write the result to an order summary table using SCD Type 2 logic to track historical changes. When customer attributes change, the system shall close the previous active row and insert a new active row with updated information.

- Business Objective:

Maintain complete historical tracking of customer-order relationships to support time-based analytics and audit requirements.

- Inputs:
- Deduplicated customer dataset from FR-CLEAN-003
- Deduplicated order dataset from FR-CLEAN-004
- Existing ordersummary table (if present) from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Processing Rules / Functional Steps:
- 1. Join customer and order datasets on CustId
- 2. For each joined record, check if a matching active row exists in ordersummary
- 3. If no matching active row exists:
 - Insert new row with IsActive = true, StartDate = current date, EndDate = null, OpTs = current timestamp
- 4. If matching active row exists and customer attributes (Name, EmailId, Region) have changed:
 - Update existing row: set IsActive = false, EndDate = current date, OpTs = current timestamp
 - Insert new row with updated customer attributes: IsActive = true, StartDate = current date, EndDate = null, OpTs = current timestamp

5. If matching active row exists and no changes detected:

- No action required

6. Write results to S3 using Apache Hudi format

• Outputs:

- Updated ordersummary table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Glue Catalog table: gen_ai_poc_databricksco.ordersummary
- Schema includes: customer fields, order fields, IsActive, StartDate, EndDate, OpTs

• Preconditions:

- Customer and order data have been cleaned and cataloged (FR-CATALOG-001, FR-CATALOG-002 complete)
- S3 location s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ is accessible
- Apache Hudi libraries are available in Glue environment

• Postconditions:

- All current customer-order relationships are marked as active (IsActive = true)
- All superseded relationships are marked as inactive (IsActive = false) with appropriate EndDate
- Historical changes are preserved

• Error / Validation Conditions:

- If join produces no results, log warning but continue processing
- If Hudi write fails, raise error and halt processing
- If customer data contains CustId not present in order data, those customers are not included in ordersummary

•

FR-INCR-001: Trigger Incremental SCD2 Updates on Customer Changes

- Title: Incremental Processing for Customer Data Changes

• Description:

The system shall detect changes in customer data and trigger incremental updates to the ordersummary table using SCD Type 2 logic, rather than full reprocessing.

• Business Objective:

Optimize processing performance and reduce compute costs by processing only changed customer records.

• Inputs:

- New or updated customer records from s3://adif-sdlc/sdlc_wizard/customerdata/

- Existing ordersummary table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Processing Rules / Functional Steps:
 1. Identify customer records that are new or have changed since last processing run
 2. For changed customers, retrieve all associated orders from sdlc_wizard_order table
 3. Apply SCD Type 2 logic (FR-SCD2-001) only to affected customer-order combinations
 4. Update ordersummary table incrementally using Hudi upsert capabilities
 5. Maintain IsActive, StartDate, EndDate, OpTs fields according to SCD Type 2 rules
- Outputs:
 - Incrementally updated ordersummary table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Preconditions:
 - Initial ordersummary table has been created (FR-SCD2-001 complete)
 - Customer data source contains change indicators or timestamps (assumed)
- Postconditions:
 - Only affected rows in ordersummary are updated
 - Historical integrity is maintained
 - OpTs reflects the incremental update timestamp
- Error / Validation Conditions:
 - If change detection fails, fall back to full processing
 - If no changes detected, skip processing and log informational message
-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Calculate and Store Customer Aggregate Spend
- Description:

The system shall aggregate order data by customer to calculate total spending and write the results to a dedicated analytics table.

- Business Objective:

Provide business users with pre-aggregated customer spending metrics for performance dashboards and customer segmentation analysis.

- Inputs:
 - Deduplicated customer dataset from FR-CLEAN-003
 - Deduplicated order dataset from FR-CLEAN-004

- Processing Rules / Functional Steps:

1. Join customer and order datasets on CustId
2. For each customer, calculate TotalAmount = SUM(PricePerUnit × Qty)
3. Group results by customer Name and Date
4. Select columns: Name, TotalAmount, Date
5. Write aggregated results to S3 location s3://adif-sdlc/analytics/customeragggregatespend/
6. Register table in Glue Data Catalog:

- Database: gen_ai_poc_databrickscoe
- Table Name: customeragggregatespend
- Schema: Name, TotalAmount, Date

- Outputs:

- customeragggregatespend table at s3://adif-sdlc/analytics/customeragggregatespend/
- Glue Catalog table: gen_ai_poc_databrickscoe.customeragggregatespend

- Preconditions:

- Customer and order data have been cleaned and deduplicated (FR-CLEAN-003, FR-CLEAN-004 complete)
- S3 location s3://adif-sdlc/analytics/customeragggregatespend/ is accessible

- Postconditions:

- Aggregate spend data is available for querying via Athena
- Each customer-date combination has a single row with total spend

- Error / Validation Conditions:

- If PricePerUnit or Qty are non-numeric, exclude those orders from aggregation and log warning
- If a customer has no orders, they will not appear in the aggregate table

-

3. Non-Functional Considerations

Monitoring and Observability

- All AWS Glue jobs shall log execution metrics to Amazon CloudWatch
- Job success/failure status shall be tracked
- Processing duration and data volume metrics shall be captured

Data Governance (Optional)

- AWS Lake Formation may be used to manage access controls and data governance policies

- If implemented, Lake Formation permissions shall align with organizational data access policies

Storage and Format

- All curated data shall be stored in S3
- SCD Type 2 tables shall use Apache Hudi format for efficient upsert operations
- Analytics tables may use Parquet or other columnar formats optimized for query performance

Query Access

- All cataloged tables shall be queryable via Amazon Athena
- Query performance optimization (partitioning, compression) is recommended but not specified in BRD
-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Description	
-----	-----	-----	
FR-INGEST-001	Source Input (AWS S3) - Customer CSV	Customer data ingestion from s3://adif-sdlc/sdlc_wizard/customerdata/	
FR-INGEST-002	Source Input (AWS S3) - Order CSV	Order data ingestion from s3://adif-sdlc/sdlc_wizard/orderdata/	
FR-CLEAN-001	Data Cleaning	Remove "Null" and NULL from customer data	
FR-CLEAN-002	Data Cleaning	Remove "Null" and NULL from order data	
FR-CLEAN-003	Data Cleaning	Remove duplicates from customer data	
FR-CLEAN-004	Data Cleaning	Remove duplicates from order data	
FR-CATALOG-001	Glue Catalog Tables	Register sdlc_wizard_customer in gen_ai_poc_databrickscoe	
FR-CATALOG-002	Glue Catalog Tables	Register sdlc_wizard_order in gen_ai_poc_databrickscoe	
FR-SCD2-001	SCD Type 2 (Hudi on S3)	Implement SCD2 for ordersummary table	
FR-INCR-001	Incremental Logic	Trigger SCD2 updates on customer changes	
FR-AGG-001	Aggregation Table	Generate customeraggregatespend analytics table	

- End of Functional Requirements Document